

SIMD 编程实验报告

2411567 陈涵

2026.05.08

1 问题描述

1.1 期末研究报告宏观问题

本课题旨在构建一套高效的混合并行口令猜测系统,需融合 MPI(跨节点任务分发)、OpenMP/Pthread 与 SIMD/GPU 等技术。核心研究问题在于:如何突破优先队列操作的锁竞争瓶颈、实现细粒度负载均衡、缓解 CPU-GPU/内存数据传输延迟,并量化分析“口令生成”与“哈希计算”在完整链路中的算力占比,从而设计最优的异构并行流水线。

1.2 本次 SIMD 编程实验子问题

本次实验聚焦于 MD5 哈希计算环节的向量化加速。由于不同口令的哈希过程相互独立、数据高度对齐且运算路径确定,天然契合 SIMD 并行范式。具体任务为:利用 SIMD 指令集(本处采用 ARM NEON)将多个独立口令打包至向量寄存器,实现单指令多数据(SIMD)同步计算;解决消息填充、状态寄存器布局转换及字节序对齐问题;实现向量化 MD5 核心函数,并与纯标量版本进行严谨的性能对比,评估实际加速比及内存带宽/指令依赖带来的性能瓶颈。

2 SIMD 算法设计与实现

2.1 算法设计思路

本次实验最终代码存放在GitHub仓库的lab2中。标准MD5算法对单个口令的处理内部存在严格的数据次序依赖,这使得我们无法在单消息内部实现指令级并行。但本选题中,系统需对大量独立口令进行哈希验证,这为任务级数据并行提供了机会。我们将采用4路并行策略,一条指令同时处理4个口令的哈希过程:

- **输入布局:** 4个独立口令 M_0, M_1, M_2, M_3 作为一批次输入
- **计算布局:** 采用结构体数组布局,即每个口令的同一状态变量打包至同一向量寄存器,不同状态变量存入不同的向量寄存器
- **输出布局:** 计算完成后转置为数组结构体布局,便于访问结果

2.2 算法实现

2.2.1 向量处理函数准备

对于原本的 MD5 算法，我们需要准备其处理函数的向量版本，以实现向对向量输入的批量处理，以下代码以 F 和 FF 的向量版本为例（具体函数实现在 md5.h 中），具体思路是使用 NEON 提供的函数替换掉原本的位运算等操作：

```

1 // 向量化位运算函数
2 static inline uint32x4_t vF(uint32x4_t x, uint32x4_t y, uint32x4_t z) {
3     return vorrq_u32(vandq_u32(x, y), vandq_u32(vmvnq_u32(x), z));
4 }
5
6 // 向量化宏
7 #define vFF(a, b, c, d, x_vec, s, ac) do { \
8     (a) = vaddq_u32(a, vF((b), (c), (d))); \
9     (a) = vaddq_u32(a, (x_vec)); \
10    (a) = vaddq_u32(a, vdupq_n_u32(ac)); \
11    (a) = vRotateLeft((a), (s)); \
12    (a) = vaddq_u32((a), (b)); \
13 } while(0)

```

2.2.2 多块数据处理

对于长度超过 64 字节的口令，MD5 需分块迭代计算。我们采用外层循环遍历最大块数、内层向量化加载的策略，确保任意长度口令的正确性

```

1 for (int block = 0; block < max_blocks; ++block) {
2     // 1. 按结构体数组布局加载当前块的 16 个消息字
3     // 2. 保存当前状态用于累加
4     // 3. 执行 64 轮向量化压缩
5     // 4. 累加: state += compressed_state
6     ...
7 }

```

2.3 复杂性分析

2.3.1 时间复杂度

- **标量版本**: 对 N 个口令, 每个口令平均 B 个 512 位块, 每块 64 轮运算, 时间复杂度为 $O(N \cdot B \cdot 64)$
- **SIMD 版本**: 4 路并行后, 理论时间复杂度降为 $O(\lceil N/4 \rceil \cdot B \cdot 64)$, 即理论加速比上限为 $4.0\times$

2.3.2 空间复杂度

- **额外空间**: 预处理需为 4 个口令分配填充缓冲区, 空间开销 $O(4 \cdot L_{max})$, 其中 L_{max} 为最长口令填充后长度

- **寄存器压力**: 需同时维护 4 组状态变量 (a, b, c, d) 及 16 个消息字向量, 共占用 $4 \times 4 + 16 = 32$ 个 128 位 NEON 寄存器, 在 ARMv8-A 架构下完全满足

2.4 算法伪代码

以下给出 md5.cpp 的 MD5Hash_SIMD() 函数的伪代码, 具体源码见提交的代码文件:

```

1 Algorithm: MD5Hash_SIMD_4way
2 Input:  inputs[4] - 4个待哈希口令字符串
3 Output: results[4][4] - 每个口令的 4 个 32 位状态字
4
5 // 批量预处理: 每个口令仅调用一次 StringProcess
6 for p <- 0 to 3 do
7   padded[p] <- StringProcess(inputs[p], &len)
8   block_count[p] <- len / 64
9   max_blocks <- max(max_blocks, block_count[p])
10 end for
11
12 // 初始化状态向量 (广播初始常数)
13 a <- vdupq_n_u32(0x67452301); b <- vdupq_n_u32(0xefcdab89)
14 c <- vdupq_n_u32(0x98badcfe); d <- vdupq_n_u32(0x10325476)
15
16 // 多块迭代处理
17 for block <- 0 to max_blocks-1 do
18   // 结构体数组布局加载: 4 个口令的同一消息字打包
19   for i <- 0 to 15 do
20     for p <- 0 to 3 do
21       if block < block_count[p] then
22         tmp[p] <- LittleEndianLoad(padded[p], block, i)
23       else
24         tmp[p] <- 0 // 短口令补零
25       end if
26     end for
27     x_vec[i] <- vld1q_u32(tmp)
28   end for
29
30   // 保存当前状态用于累加
31   (a_save, b_save, c_save, d_save) <- (a, b, c, d)
32
33   // 64 轮向量化压缩 (按标准 MD5 顺序)
34   for round <- 0 to 63 do
35     apply vFF/vGG/vHH/vII macro with x_vec[round_index]
36   end for
37
38   // 状态累加
39   a <- vaddq_u32(a_save, a); b <- vaddq_u32(b_save, b)
40   c <- vaddq_u32(c_save, c); d <- vaddq_u32(d_save, d)
41 end for
42
43 // 字节序转换 + 布局转置
44 for p <- 0 to 3 do
45   results[p][0] <- ByteSwap_ExtractLane(a, p)
46   results[p][1] <- ByteSwap_ExtractLane(b, p)

```

```

47     results[p][2] <- ByteSwap_ExtractLane(c, p)
48     results[p][3] <- ByteSwap_ExtractLane(d, p)
49 end for
50
51 // 内存清理
52 for p <- 0 to 3 do
53     delete[] padded[p]
54 end for

```

3 实验及结果分析

3.1 实验结果

通过在 OpenEuler 的服务端对于标量版本与 SIMD 版本的代码，分别在 -O0、-O1、-O2 的优化级别下调用 g++ 编译命令，再使用 test.sh 提交运行，并使用 perf 对部分指标加以关注，得到以下六组数据：

表 1: 不同编译器优化等级下的 perf 性能指标与加速比对比（1000 万条口令）

优化等级	版本	Cycles ($\times 10^9$)	Instructions ($\times 10^9$)	IPC	Cache Miss Rate	Hash Time (s)	加速比
-O0	标量	327.06	356.00	1.09	0.601%	9.56	—
	SIMD	339.83	353.97	1.04	0.575%	15.60	0.61 $\times$
-O1	标量	102.20	60.43	0.59	4.338%	3.16	—
	SIMD	96.27	55.08	0.57	4.563%	2.09	1.51 $\times$
-O2	标量	98.95	55.93	0.57	5.650%	3.14	—
	SIMD	93.46	49.16	0.53	5.718%	1.77	1.78 $\times$

图 1: -O2 优化等级下 perf 原始输出对比 (1000 万条口令)

<pre> Guesses generated: 10106852 Guess time:0.68881seconds Hash time:3.14135seconds Train time:30.7213seconds Performance counter stats for './main': 98,951,595,145 cycles:u 55,927,489,682 instructions:u # 0.57 insn per cycle 17,321,623,434 cache-references:u 978,719,231 cache-misses:u # 5.650 % of all cache refs 39.129638264 seconds time elapsed 38.597155000 seconds user 0.422741000 seconds sys </pre>	<pre> SIMD time: Guess time:0.625847seconds Hash time:1.76828seconds Train time:30.0227seconds Performance counter stats for './main': 93,464,753,526 cycles:u 49,163,375,886 instructions:u # 0.53 insn per cycle 16,252,578,468 cache-references:u 929,343,983 cache-misses:u # 5.718 % of all cache refs 37.049698173 seconds time elapsed 36.552740000 seconds user 0.378746000 seconds sys </pre>
---	---

(a) 标量版本 (Hash time: 3.14s)

(b) SIMD 版本 (Hash time: 1.77s, 加速比 1.78×)

注:

由于终端信息保存有长度限制, 此处仅附上-O2 下的原始 perf 结果截图

3.2 结果分析

3.2.1 关键指标解读

1. 指令数与 IPC 变化趋势:

- -O0 时标量版本 IPC 更高 (1.09vs1.04), 说明未优化下 32 位通用寄存器的调度效率优于 128 位 NEON 向量寄存器, 后者因函数调用开销导致流水线停顿增加。
- -O1/-O2 时两者 IPC 均下降至 0.53~0.59, 反映编译器优化后经过循环展开与指令融合, 指令密度有所提升, 但 MD5 函数数据的轮次依赖还是限制了指令级并行的进一步提升。
- SIMD 版本在 -O2 下指令数减少 12%, 印证了单条 NEON 指令 (如 vaddq_u32) 可替代约 4 条标量指令的理论预期。

2. 缓存行为分析:

- 未命中率随优化等级上升, 是因为 -O1/-O2 下循环展开、预取等优化增加了缓存引用频率, 导致分母增大。
- SIMD 与标量的 miss rate 差异始终 <0.1 个百分点, 说明 4 路并行并未引发显著的缓存颠簸, 口令数据本身具有良好的空间局部性。
- 尽管 4 路并行使单次数据加载量增大 4 倍, 但缓存未命中绝对值仍 <6%, 说明内存带宽并非主要瓶颈。

3. 周期数与壁钟时间的解耦现象:

在-O0 下, 周期数从标量的 327.06×10^9 变为了 SIMD 的 339.83×10^9 , 仅有 3.9% 的增加, 但是关注哈希时间这一指标, 标量为 9.56s, SIMD 为 15.60s, 却有 63% 的增加, 说明这两个指标之间的解耦或为-O0 下负加速现象的关键原因。

经过查询资料, 得知 perf 的 cycles:u 仅统计用户态 CPU 执行周期, 不包含内存分配器的系统调用等待, 页面错误导致的缺页中断和函数调用上下文切换的流水线刷新。而-O0 下 SIMD 版本的 4 次 StringProcess 与内存分配会产生大量堆调用, 这些系统态延迟未被 cycles:u 捕获, 但显著拉长了壁钟时间。

3.2.2 为什么能够加速？

MD5 哈希环节之所以能实现有效加速，核心在于其**数据独立性、计算确定性、内存对齐性**与 SIMD 硬件特性的契合。本实验通过合理的布局设计、向量函数封装及-O2 编译器优化协同，成功将理论并行度转化为 $1.78\times$ 的实测性能增益。该结果成功验证了指导书中“MD5 非常适合 SIMD 并行”的论断。

3.2.3 编译选项对加速比的影响

由表可知，随着编译时选择的加速等级的上升，加速比由 $0.61\times$ 提高到 1.51 ，再提升到 $1.78\times$ 。经查询得知，SIMD 加速高度依赖编译器优化。-O0 下负加速是典型现象，而-O1 与-O2 的加速比符合 4 路并行的理论预期（考虑串行部分后）。

3.2.4 -O0 时的负加速归因

在 -O0 配置下，SIMD 版本哈希耗时 (15.60s) 较标量版本 (9.56s) 增加 63%，加速比仅为 $0.61\times$ ，起到了反向加速的效果。经过查询资料得知此现象出现并非算法问题，而是-O0 优化等级下，对编译器高度依赖的 SIMD 加速的典型现象。结合 perf 数据与代码结构进行分析，得知可能的归因如下：

1. **函数内联失效**：-O0 下 `static inline` 修饰的向量处理函数 (`vF`, `vRotateLeft`, `vByteSwap` 等) 未被编译器内联。每次宏展开均产生真实函数调用，涉及 128 位 NEON 向量寄存器的栈操作。标量版本虽也未优化，但 32 位通用寄存器 (GPR) 的调用开销远小于向量寄存器，导致 SIMD 版本流水线停顿增加 (IPC 从 1.09 降至 1.04)。
2. **批处理固定开销未分摊成功**：SIMD 版本引入的 4 次 `StringProcess` 调用、内存分配/释放、结构体数组布局加载与之后的布局转置在会-O0 下产生大量堆管理例程调用。4 路并行的计算收益被预处理与布局转换的串行延迟完全覆盖。例如，`new Byte[]` 在-O0 下会调用完整的 `malloc`，而-O2 下也许会被优化为栈分配或更合理的内存池复用。
3. **寄存器分配保守导致溢出**：-O0 的寄存器分配策略极其保守，向量运算中间结果频繁溢出至栈内存。标量版本仅操作 32 位通用寄存器，内存往返路径更短。壁钟时间差异主要来源于内存分配延迟与流水线等待，而非纯指令执行周期（用户态周期数仅增加 3.9%）。考虑到此次测试中千万级别的数据量，这个原因很有可能是关键。
4. **编译器自动向量化缺失**：-O0 下 GCC 不会对标量代码进行任何自动向量化，但也不会对手动 SIMD 代码进行优化。相比之下，-O1/-O2 下编译器会对标量版本进行部分自动向量化，缩小了手动 SIMD 的相对优势，但同时也消除了-O0 下的系统性开销。

该现象符合向量化编程的典型特征：SIMD 加速高度依赖编译器优化以消除内联开销、循环展开与寄存器化。在-O2 的优化等级下，上述瓶颈最终被有效缓解，实测加速比恢复至 $1.78\times$ 。-O0 下的数据成功验证了编译器优化对 SIMD 效能的决定性影响。